CS 61A
Past Exam Practice — Linked Lists and Trees

Your Name: _____

Your Student ID: _____

Room you are taking the exam in: _____

Name and SID of the person to your **left**: _____

Name and SID of the person to your **right**: _____

You have 110 minutes. There are 8 questions of varying credit. (43 points total)

Question	1	2	3	4	5	6	7	8	Total
Points	7	5	5	4	5	6	6	5	43

For questions with **circular bubbles**, you may select only one choice.

- ○ Unselected option (Completely unfilled)
- ◎ Don't do this (it will be graded as incorrect)
- ● Only one selected option (completely filled)

For questions with **square checkboxes**, you may select one or more choices.

- ■ You can select
- ■ multiple squares
- ☑ (Don't do this)

Make sure to **write your student ID at the top of each page** in the provided blank.

Anything you write outside the answer boxes or you ~~cross out~~ will not be graded. If you write multiple answers, your answer is ambiguous, or the bubble/checkbox is not entirely filled in, we will grade the worst interpretation. For coding questions with blanks, you may write at most one statement per blank and you may not use more blanks than provided.

As a member of the UC Berkeley community, I act with honesty, integrity, and respect for others. I will follow the rules of this exam. I will not use any other unauthorized materials or devices during this exam. I will not communicate with anyone other than the proctors during this exam. I will not share any information about the exam with anyone until after the end of the exam period.

Acknowledge that you have read and agree to the honor code above and **sign your name below**:

_____

This content is protected and may not be shared, uploaded, or distributed.

Reference: New `Link` and `Tree` Types

Use these definitions for every question in this worksheet.

```
from __future__ import annotations
from dataclasses import dataclass, field

type LinkedList[T] = Link[T] | tuple[()]

@dataclass
class Link[T]:
    first: T
    rest: LinkedList[T] = ()

    def __str__(self):
        return format_link(self)

def format_link(s: Link):
    string = '(' + str(s.first)
    remaining = s.rest
    while isinstance(remaining, Link):
        string += ' ' + str(remaining.first)
        remaining = remaining.rest
    assert remaining == (), f'{s!r} is not a LinkedList'
    return string + ')'

@dataclass
class Tree[T]:
    label: T
    branches: list[Tree[T]] = field(default_factory=list)

    def __str__(self):
        return format_tree(self)

def format_tree(t: Tree, indent='') -> str:
    assert isinstance(t, Tree), f'{t!r} is not a Tree'
    assert isinstance(t.branches, list), f'branches of {t!r} is not a list'
    string = indent + str(t.label)
    for b in t.branches:
        string += '\n' + format_tree(b, indent + '  ')
    return string

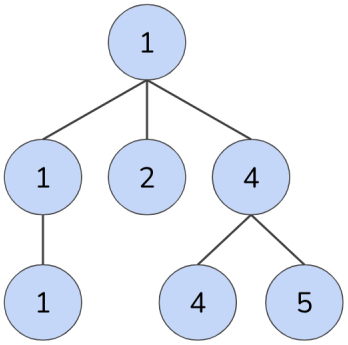
def is_leaf(t: Tree) -> bool:
    return not t.branches
```


This content is protected and may not be shared, uploaded, or distributed.

Q1 *All Treeils Lead to Rome* **(7 points)**

Original source: Summer 2023 - Midterm - Question 6(a) | **Level:** Regular

Definition: A **Treeil** is a tree in which *exactly* one of the child node(s) of each non-leaf node has a label equal to the label of the node. Alternatively, a treeil is a tree where every node has a singular path from the node to a leaf consisting entirely of nodes with the same label. Here is an example of a valid treeil:



Treeil Trial

Implement `is_treeil`, which determines whether a tree `t` is a treeil.

```
def is_treeil(t):
    """Return whether t is a treeil. A treeil is a tree where exactly one of
    the branches of each non-leaf node has a label equal to the label of the node.

    >>> t1 = Tree(1)
    >>> is_treeil(t1)
    True
    >>> t2 = Tree(1, [Tree(2), Tree(3)])
    >>> is_treeil(t2)
    False
    >>> t3 = Tree(1, [Tree(1, [Tree(1), Tree(3)]), Tree(2)])
    >>> is_treeil(t3)
    True
    >>> t4 = Tree(1, [Tree(1, [Tree(1), Tree(1)]), Tree(2)])
    >>> is_treeil(t4)
    False
    >>> t5 = Tree(2, [Tree(3, [Tree(1)]), Tree(2, [Tree(2)])])
    >>> is_treeil(t5)
    False
    """
    if _________:
       # (a)
        return True
    else:
        match_one = _________([b for b in t.branches if _________]) == 1
                    # (b)                               # (c)
        return _________ and _________([_________ for b in t.branches])
               # (d)       # (e)      # (f)
```


This content is protected and may not be shared, uploaded, or distributed.

(Question 1 continued…)

Q1.1 (1 point) Fill in blank (a). Select **all** that apply.

- ☐ `True`
- ☐ `False`
- ☐ `is_leaf(t)`
- ☐ `t.branches`
- ☐ `not t.branches`
- ☐ `t.label`
- ☐ `not t.label`
- ☐ `t.branches == []`

Q1.2 (1 point) Fill in blank (b).

- ◯ `sum`
- ◯ `max`
- ◯ `min`
- ◯ `len`
- ◯ `any`
- ◯ `all`

Q1.3 (2 points) Fill in blank (c).

Q1.4 (1 point) Fill in blank (d).

Q1.5 (1 point) Fill in blank (e).

- ◯ `sum`
- ◯ `max`
- ◯ `min`
- ◯ `len`
- ◯ `any`
- ◯ `all`

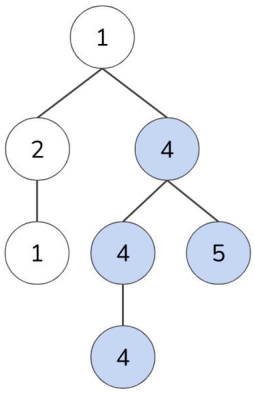
Q1.6 (1 point) Fill in blank (f).

This content is protected and may not be shared, uploaded, or distributed.

Q2 *Treeil Trail* **(5 points)**

Original source: Summer 2023 - Midterm - Question 6(b) | **Level:** Regular

Implement `max_treeil` which takes in a tree `t` and determines the highest number of nodes in a treeil that is a subtree of `t`. You may assume is_treeil is implemented correctly from the previous question. The following diagram displays a tree `t`, which is not a treeil. The treeil subtree of `t` with the highest number of nodes is highlighted. Calling `max_treeil` on this tree `t` should give a result of 4.




```
1  def max_treeil(t):
2      """
3      Given a tree (that is not necessarily a treeil), returns
4      the highest number of nodes in a treeil in the tree.
5
6      >>> max_treeil(Tree(1))
7      1
8      >>> max_treeil(Tree(1, [Tree(2), Tree(3)]))
9      1
10     >>> t3 = Tree(1, [Tree(1, [Tree(1), Tree(3)]), Tree(2)])
11     >>> max_treeil(t3)
12     5
13     >>> t4 = Tree(1, [Tree(1, [Tree(1), Tree(1)]), Tree(2)])
14     >>> max_treeil(t4)
15     1
16     >>> t5 = Tree(2, [Tree(3, [Tree(1)]), Tree(2, [Tree(2)])])
17     >>> max_treeil(t5)
18     2
19     """
20     if _________:
21        # (a)
22         return 1
23     elif _________:
24          # (b)
25         return 1 + _________
26                    # (c)
27     else:
28         return max([_________ for b in t.branches])
29                     # (d)
```

Q2.1 (1 point) Fill in blank (a).

- ◯ `is_treeil(t)`
- ◯ `is_treeil(t) or is_leaf(t)`
- ◯ `not is_treeil(t)`
- ◯ `not is_treeil(t) and is_leaf(t)`
- ◯ `is_leaf(t)`

This content is protected and may not be shared, uploaded, or distributed.

(Question 2 continued…)

Q2.2 (1.5 points) Fill in blank (b).

Q2.3 (1 point) Fill in blank (c).

- ◯ `max([max_treeil(b) for b in t.branches])`
- ◯ `len([max_treeil(b) for b in t.branches])`
- ◯ `sum([max_treeil(b) for b in t.branches])`
- ◯ `sum([b for b in t.branches if is_treeil(b)])`
- ◯ `len([b for b in t.branches if is_treeil(b)])`

Q2.4 (1.5 points) Fill in blank (d).

This content is protected and may not be shared, uploaded, or distributed.

Q3 *Prefixes* (5 points)

Original source: Spring 2023 - Midterm 2 - Question 3 | **Level:** Regular

Implement `tens`, which takes a **non-empty** linked list of numbers `s` represented as a `Link` instance. It prints all of the prefix sums of `s` that are multiples of 10 in increasing order of the length of the prefix.

The `Link` class appears on the reference page at the beginning of this worksheet.

```
def tens(s):
    """Print all prefix sums of Link s that are multiples of ten.

    >>> tens(Link(3, Link(9, Link(8, Link(10, Link(0, Link(14, Link(6)))))))
    20
    30
    30
    50
    """

    def f(suffix, total):

        if total % 10 == 0:

            print(total)

        if _________:
              # (d)

            _________
               # (e)


    _________
       # (f)
```

Q3.1 (1 point) Fill in blank (d).

- ◯ `isinstance(s, Link)`
- ◯ `isinstance(suffix, Link)`
- ◯ `isinstance(s.rest, Link)`
- ◯ `isinstance(suffix.rest, Link)`
- ◯ `isinstance(s.rest.rest, Link)`
- ◯ `isinstance(suffix.rest.rest, Link)`

Q3.2 (2 points) Fill in blank (e).

Q3.3 (2 points) Fill in blank (f).

This content is protected and may not be shared, uploaded, or distributed.

Q4 *Trees* **(4 points)**

Original source: Spring 2024 - Final - Question 5(a) | **Level:** Regular

Implement `count_ones`, which takes a `Tree` instance `t`. It returns the number of nodes in `t` that have one child. The `Tree` class is defined on the reference page at the beginning of this worksheet.

```
def count_ones(t):
    """Return the number of nodes that have one child.

    >>> count_ones(Tree(1, [Tree(2), Tree(3, [Tree(4)])]))
    1
    >>> count_ones(Tree(1, [Tree(2), Tree(3), Tree(4)]))
    0
    >>> count_ones(Tree(1, [Tree(2, [Tree(3, [Tree(4)]), Tree(5, [Tree(6)])])]))
    3
    """
    count = sum([ count_ones(_______) for b in t.branches if _______ ])
                               # (a)                              # (b)
    if _______:
         # (c)
        return count + 1
    else:
        return count
```

Q4.1 (1 point) Fill in blank (a).

- ◯ `b`
- ◯ `b.branches`
- ◯ `t`
- ◯ `t.branches`

Q4.2 (1 point) Fill in blank (b).

- ◯ `True`
- ◯ `len(b.branches) == 1`
- ◯ `is_leaf(b)`
- ◯ `len(t.branches) == 1`
- ◯ `is_leaf(t)`

Q4.3 (2 points) Fill in blank (c).

- ◯ `True`
- ◯ `len(t.branches) == 1`
- ◯ `is_leaf(b)`
- ◯ `all([len(b.branches) == 1 for b in t.branches])`
- ◯ `is_leaf(t)`
- ◯ `any([len(b.branches) == 1 for b in t.branches])`
- ◯ `len(b.branches) == 1`

This content is protected and may not be shared, uploaded, or distributed.

Q5 *How Long Is This Exam?* **(5 points)**

Original source: Fall 2024 - Midterm 2 - Question 5(a) | **Level:** Regular

Longer

Implement `longer`, which takes two linked lists of numbers `s` and `t`. It returns the longer of the two. If they have the same length, it returns `s`.

A linked list is either a `Link` instance or `()`. The `Link` class is on the reference page at the beginning of this worksheet.

```
def longer(s, t):
    """Return the longer linked list, s or t. (Same length? Return s.)

    >>> print(longer(Link(2, Link(3)), ()))
    (2 3)
    >>> print(longer(Link(2, Link(3)), Link(4, Link(5))))
    (2 3)
    >>> print(longer(Link(2, Link(3)), Link(4, Link(5, Link(6, Link(7))))))
    (4 5 6 7)
    >>> longer((), ()) == ()
    True
    """
    a, b = s, t
    while isinstance(b, Link):
        if _______:
           # (a)
            return _______
                   # (b)

        _______
        # (c)
    return _______
           # (d)
```

Q5.1 (2 points) Fill in blank (a).

Q5.2 (1 point) Fill in blank (b).

- ○ `a`
- ○ `b`
- ○ `s`
- ○ `t`
- ○ `longer(a, b)`
- ○ `longer(s, t)`

Q5.3 (1 point) Fill in blank (c).

- ○ `a = a.rest`
- ○ `a = s.rest`
- ○ `b = b.rest`
- ○ `b = t.rest`
- ○ `a, b = a.rest, b.rest`
- ○ `a, b = s.rest, t.rest`

Q5.4 (1 point) Fill in blank (d).

- ○ `a`
- ○ `b`
- ○ `s`
- ○ `t`
- ○ `longer(a, b)`
- ○ `longer(s, t)`

This content is protected and may not be shared, uploaded, or distributed.

Q6 *Longest* **(6 points)**

Original source: Fall 2024 - Midterm 2 - Question 5(b) | **Level:** Challenge

Implement `longest`, which takes a linked list of postive integers `s` and a positive integer `n`. It returns the longest sublist of `s` with elements that sum to a number less than or equal to `n`. **Do not mutate `s`.** In case of a tie, return any of the longest sublists whose sum is `n` or less. Assume `longer` is implemented correctly.

A sublist of a linked list `s` is a linked list with some (or none or all) of the elements of `s` in order.

Assume the sum of the elements of `()` is 0. `()` is a sublist of any linked list.

```
def longest(s, n):
    """Return the longest sublist of s that sums to n or less.

    >>> print(longest(Link(5, Link(1, Link(3, Link(4, Link(2, Link(7)))))), 7))
    (1 3 2)
    >>> print(longest(Link(5, Link(1, Link(3, Link(4, Link(2, Link(7)))))), 70))
    (5 1 3 4 2 7)
    >>> longest(Link(3, Link(4, Link(5))), 2) == ()
    True
    """
    if not isinstance(s, Link):
        return s
    t = _______
        # (e)
    if _______:
       # (f)
        return longer(_______, t)
                      # (g)
    else:
        return t
```

Q6.1 (1 point) Fill in blank (e).

- ◯ `longest(s, n)`
- ◯ `longest(s.rest, n)`
- ◯ `longest(s, n - s.first)`
- ◯ `longest(s.rest, n - s.first)`

Q6.2 (2 points) Fill in blank (f).

Q6.3 (3 points) Fill in blank (g).

This content is protected and may not be shared, uploaded, or distributed.

Q7 *Maximum Exponen-tree-ation* **(6 points)**

Original source: Summer 2021 - Midterm - Question 4 | **Level:** Challenge

For a list of integers `values`, define the **maximum exponentiation** of `values` to be the largest single number that can be achieved by repeatedly combining pairs of consecutive elements from `values` through exponentiation. Two possible exponentiations for the list `[3, 1, 2, 3]` are shown below:

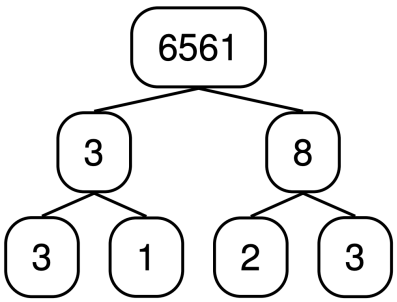
```
(3 ** 1) ** (2 ** 3) = 6561
(3 ** (1 ** 2)) ** 3 = 27
```

Of all the exponentiations of `[3, 1, 2, 3]`, the maximum exponentiation is `6561`, which can be calculated through the below steps:

```
[3, 1, 2, 3]
[3 ** 1, 2, 3]
[3, 2, 3]
[3, 2 ** 3]
[3, 8]
[3 ** 8]
6561
```

Implement `exp_tree`, which accepts a non-empty list of integers `values` and returns a tree whose label is the maximum exponentiation of `values`. Given that the maximum exponentiation is `b ** n` for some two integers `b` and `n`, the two branches of this tree should be the `exp_trees` of the sublists of `values` used to exponentiate to `b` and `n`, respectively. You may assume that `values` has a unique exponentiation tree.

For example, below is a drawing of the `exp_tree` for `values = [3, 1, 2, 3]`:



The function should assume the `Tree` definition on the reference page at the beginning of this worksheet.

Note: Exponentials in Python are expressed as `base ** exponent`.

This content is protected and may not be shared, uploaded, or distributed.

```
def exp_tree(values):
    """
    Returns the exponential tree that can be made from VALUES
    with the greatest possible root label

    >>> print(exp_tree([5]))
    5
    >>> print(exp_tree([3, 2]))
    9
      3
      2
    >>> print(exp_tree([2, 3, 2]))
    512
      2
      9
        3
        2
    >>> lst = [3, 1, 2, 3]
    >>> print(exp_tree(lst))
    6561
      3
        3
        1
      8
        2
        3
    """
    if __________:
        # (a)
       return __________
                # (b)
    else:
        def tree_at_split(i):
            base = exp_tree(__________)
                             # (c)
            exponent = exp_tree(__________)
                                # (d)
            return Tree(__________, [base, exponent])
                        # (e)
        trees = [tree_at_split(i) for i in range(1, len(values))]
        return max(trees, key=__________)
                                # (f)
```

Q7.1 (1 point) Which of these could fill in blank (a)?

- ◯ `len(values) == 0`
- ◯ `len(values) == 1`
- ◯ `values`
- ◯ `len(values) == 2`
- ◯ `values[0] < values[1]`
- ◯ `len(values) <= 2`
- ◯ `len(values) <= 3`

Q7.2 (1 point) Which of these could fill in blank (b)?

- ◯ `0`
- ◯ `Tree(0)`
- ◯ `values[0]`
- ◯ `Tree(values[0])`
- ◯ `values[1]`

This content is protected and may not be shared, uploaded, or distributed.

(Question 7 continued…)

- ◯ `Tree(values[1])`
- ◯ `values[0] ** values[1]`
- ◯ `Tree(values[0] ** values[1])`
- ◯ `values[1] ** values[0]`
- ◯ `Tree(values[1] ** values[0])`

Q7.3 (1 point) What line of code could go in blank (c)?

Q7.4 (1 point) What line of code could go in blank (d)?

Q7.5 (1 point) What line of code could go in blank (e)?

Q7.6 (1 point) What line of code could go in blank (f)?

This content is protected and may not be shared, uploaded, or distributed.

Q8 *Skipping Around* **(5 points)**

Original source: Summer 2021 - Midterm - Question 2(b) | **Level:** Challenge

A skip list is defined as a sublist of a list such that each element in the sublist is non adjacent in the original list. For original list `[5, 6, 8, 2]`, the lists `[5, 8]`, `[5, 2]`, `[6, 2]`, `[5]`, `[6]`, `[8]`, `[2]`, `[]` are all skip lists of the original list. The empty list is always a skip list of any list.

Given a list `int_lst` of unique integers, return a list of all unique skip lists of `int_lst` where **each skip list contains integers in strictly increasing order**. The order in which the skip lists are returned does not matter.

```
1  def list_skipper(int_lst):
2      """
3      >>> list_skipper([5,6,8,2])
4      [[5, 8], [5], [6], [8], [2], []]
5      >>> list_skipper([1,2,3,4,5])
6      [[1, 3, 5], [1, 3], [1, 4], [1, 5], [1], [2, 4], [2, 5], [2], [3, 5], [3], [4], [5], []]
7      >>> list_skipper([])
8      [[]]
9      """
10     if len(int_lst) == 0:
11         return _________________
12                     # (a)
13     with_first = ____________________
14                       # (b)
15     without_first = ____________________
16                         # (c)
17     with_first = [ _________ for x in with_first if x == [] or _____________]
18                    # (d)                                          # (e)
19     return with_first + without_first
```

Q8.1 (1 point) What line of code could go in blank (a)?

Q8.2 (1 point) Which of these could fill in blank (b)?

- ◯ `list_skipper(int_lst)`
- ◯ `list_skipper(int_lst[0])`
- ◯ `list_skipper(int_lst[1:])`
- ◯ `list_skipper(int_lst[:-1])`
- ◯ `list_skipper(int_lst[2:])`

Q8.3 (1 point) What line of code could go in blank (c)?

Q8.4 (1 point) Which of these could fill in blank (d)?

- ◯ `x`
- ◯ `[int_lst[0]] + x`
- ◯ `x[0]`
- ◯ `int_lst.append(x)`
- ◯ `[x[0]] + int_lst`
- ◯ `x.append(int_lst)`

Q8.5 (1 point) Which of these could fill in blank (e)?

- ◯ `x[0] < int_lst[0]`
- ◯ `x[0] > int_lst[0]`
- ◯ `x < int_lst[0]`

This content is protected and may not be shared, uploaded, or distributed.